



Centro de Educação Superior de Brasília
Centro Universitário Instituto de Educação Superior de Brasília

Professor: José Reginaldo de Sousa Mendes Junior

Disciplina: Projeto Integrador

Fase 1

Projeto Integrador

Aluno: Alanna Tomaz de Aquino Martins **Matrícula:** 2412130055

Aluno: Sofia Dios **Matrícula:** 2412130138

Relatório Técnico — Fase I

1. Descrição do Problema

O objetivo desta fase é estabelecer um baseline experimental para análise de desempenho de algoritmos de busca. Para isso, foi implementada a leitura de um dataset real de produtos, seu armazenamento em vetor com alocação dinâmica de memória, e a execução de buscas sequenciais por ID com medição rigorosa de tempo de execução.

A busca sequencial servirá como ponto de referência comparativo na Fase II, onde será implementada uma Tabela Hash. A comparação entre as duas abordagens será o núcleo da análise final do projeto.

2. Caracterização do Dataset

Atributo	Valor
Nome do arquivo	<code>dataset4.csv</code>
Total de registros carregados	400.009
Campos por registro	id (int), nome (string[51]), categoria (string[31]), valor (float)

Exemplo de registro 371601, MSI Placa de Vídeo RTX Inspiron 2227,
Informática, 11784.25

A estrutura utilizada no programa é:

```
typedef struct {  
    int id;  
    char nome[51];  
    char categoria[31];  
    float valor;  
} Produto;
```

O arquivo foi lido com verificação de erros (arquivo inexistente, linhas com formato inválido) e armazenado em um vetor com alocação dinâmica de memória, utilizando `malloc` para a alocação inicial e `realloc` para expansão conforme necessário.

3. Metodologia de Testes

3.1 Protocolo Seguido

1. O vetor foi completamente carregado antes do início de qualquer medição.
2. Foram sorteados 1.000 IDs aleatoriamente, divididos em 4 blocos de 250:
 - Início — IDs sorteados no primeiro quarto do vetor (posições 0 a 100.002)
 - Meio — IDs sorteados entre o primeiro e terceiro quartos (posições 100.002 a 300.006)
 - Final — IDs sorteados no último quarto do vetor (posições 300.006 a 400.009)
 - Inexistente — ID -999, garantidamente ausente no vetor
3. Os IDs sorteados foram salvos em arquivo (`amostras.txt`) para garantir que as mesmas amostras sejam usadas em todas as execuções, tornando os resultados reproduzíveis.
4. O protocolo foi repetido 3 vezes (3 rodadas) com os mesmos IDs.
5. O tempo foi medido com `clock()` da biblioteca `<time.h>`, separadamente para cada bloco dentro de cada rodada.

3.2 Fórmula do Tempo Médio

Tempo médio por busca = Tempo total do bloco / Número de buscas do bloco

4. Resultados Obtidos

Rodada 1

Bloco	Busca s	Média por busca (s)
Início	250	0,000276
Meio	250	0,001428
Final	250	0,003164
Inexistente	250	0,003244
Total da rodada	1.000	0,002028

Rodada 2

Bloco	Busca s	Média por busca (s)
Início	250	0,000276
Meio	250	0,001304
Final	250	0,002624
Inexistente	250	0,003444
Total da rodada	1.000	0,001912

Rodada 3

Bloco	Busca s	Média por busca (s)
Início	250	0,000304
Meio	250	0,001204
Final	250	0,002512
Inexistente	250	0,002960
Total da rodada	1.000	0,001745

Resumo Final

	Média por busca (s)
Rodada 1	0,002028
Rodada 2	0,001912
Rodada 3	0,001745
Média geral	0,001895

4.1 Print da máquina rodando localmente

```
alann@AlannaTomazNOTE MINGW64 ~/pi (alanna)
$ gcc -o fase1 src/main.c

=====
Inicio      : 250 buscas | Media: 0.000288 s/busca
Meio        : 250 buscas | Media: 0.001416 s/busca
Final       : 250 buscas | Media: 0.002704 s/busca
Inexistente : 250 buscas | Media: 0.003636 s/busca
-----
Tempo total da rodada : 2.011000 s
Media por busca       : 0.002011 s

=====
Rodada 2
=====
Inicio      : 250 buscas | Media: 0.000268 s/busca
Meio        : 250 buscas | Media: 0.001180 s/busca
Final       : 250 buscas | Media: 0.003304 s/busca
Inexistente : 250 buscas | Media: 0.003272 s/busca
-----
Tempo total da rodada : 2.006000 s
Media por busca       : 0.002006 s

=====
Rodada 3
=====
Inicio      : 250 buscas | Media: 0.000280 s/busca
Meio        : 250 buscas | Media: 0.001388 s/busca
Final       : 250 buscas | Media: 0.003024 s/busca
Inexistente : 250 buscas | Media: 0.003236 s/busca
-----
Tempo total da rodada : 1.982000 s
Media por busca       : 0.001982 s

=====
Resumo Final
=====
Media de tempo - Rodada 1 : 0.002011 s/busca
Media de tempo - Rodada 2 : 0.002006 s/busca
Media de tempo - Rodada 3 : 0.001982 s/busca
Media geral              : 0.002000 s/busca
=====

Testes concluidos com sucesso.
```

5. Análise dos Resultados

5.1 Comportamento Observado

Os resultados demonstram claramente o comportamento linear da busca sequencial. Observando os tempos por bloco, é possível perceber uma progressão direta entre a

posição do elemento no vetor e o tempo necessário para encontrá-lo: buscas no início foram aproximadamente 10 vezes mais rápidas do que buscas no final, o que é coerente com a complexidade $O(n)$ do algoritmo.

O bloco de elementos inexistentes apresentou o maior tempo médio em todas as rodadas, pois o algoritmo percorre o vetor inteiro sem encontrar o ID buscado — representando sempre o pior caso da busca sequencial.

5.2 Efeito de Aquecimento do Cache

Nota-se que a Rodada 1 foi a mais lenta (0,002028 s/busca) e a Rodada 3 a mais rápida (0,001745 s/busca). Isso ocorre devido ao efeito de aquecimento do cache do processador: na primeira rodada, os dados precisam ser buscados na memória RAM; nas rodadas seguintes, parte dos dados já está no cache, acelerando os acessos. Esse é o motivo pelo qual o protocolo utiliza 3 rodadas e calcula a média — para diluir esse efeito e obter um resultado mais representativo.

5.3 Relação entre Posição e Tempo de Busca

Bloco	Média Rodada 1	Média Rodada 2	Média Rodada 3
Início	0,000276 s	0,000276 s	0,000304 s
Meio	0,001428 s	0,001304 s	0,001204 s
Final	0,003164 s	0,002624 s	0,002512 s
Inexistente	0,003244 s	0,003444 s	0,002960 s

A tabela confirma que o tempo cresce proporcionalmente à posição do elemento — exatamente o comportamento esperado de um algoritmo $O(n)$.

5.4 Limitações da Busca Sequencial

Limitação	Descrição
Complexidade $O(n)$	O tempo cresce linearmente com o número de elementos
Pior caso custoso	Elementos inexistentes exigem a varredura completa do vetor

Não aproveita ordenação	Mesmo que o vetor estivesse ordenado, a busca sequencial não tira vantagem disso
Não escalável	Para milhões de registros, o tempo se torna proibitivo em aplicações reais

6. Conclusão Preliminar

A busca sequencial cumpre seu papel como baseline: é simples de implementar, funciona corretamente e seus resultados são previsíveis. Com 400.009 registros e uma média geral de 0,001895 segundos por busca, fica evidente a limitação da abordagem para grandes volumes de dados.

Na Fase II, a implementação de uma Tabela Hash deverá reduzir drasticamente esse tempo, aproximando-se de $O(1)$ por busca, independentemente da posição do elemento no conjunto de dados.